CIS 3300 · INHERITANCE AND POLYMORPHISM

Polymorphism and Casting

Part 2 of 4: one variable, many kinds of objects, and how Java decides which method runs

An object never changes form; the reference does

Polymorphism: a variable of a superclass type can refer to objects of many subclasses, and each object still responds to method calls in its own way.

```
Employee someone = new Manager("Grace", 90000, 12000);
```

Compiles

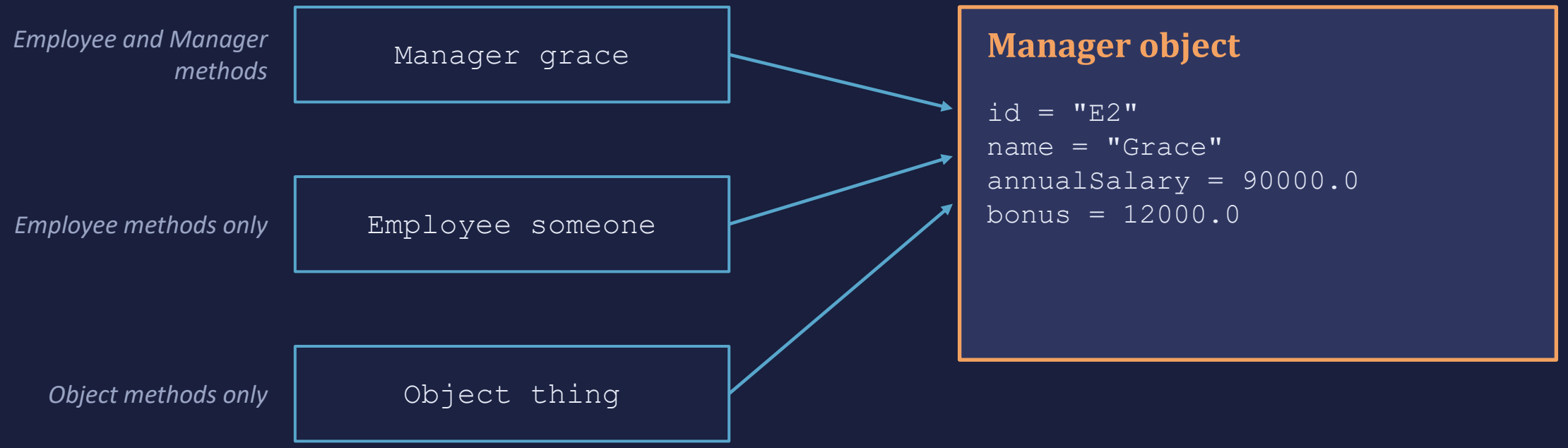
```
Manager boss = new Employee("Ada", 60000);
```

Does not compile

Substitution principle: a subclass object can be used anywhere its superclass is expected. Every manager is an employee; not every employee is a manager.

Java has two kinds of polymorphism, compile-time and runtime. Slide 4 compares them.

The declared type limits; the actual type decides




```
someone.getMonthlyPay();    // runs Manager's version: 8500.0
someone.getBonus();         // error: cannot find symbol
```

Compile-time polymorphism and runtime polymorphism

```
static void printBadge(Employee e) { ... "STAFF" ... }
static void printBadge(Manager m)  { ... "MANAGER" ... }

Manager grace = new Manager("Grace", 90000, 12000);
Employee someone = grace;      // the same object
printBadge(grace);            // Badge: MANAGER Grace
printBadge(someone);          // Badge: STAFF   Grace
```

Compile time (static, early binding): overloading

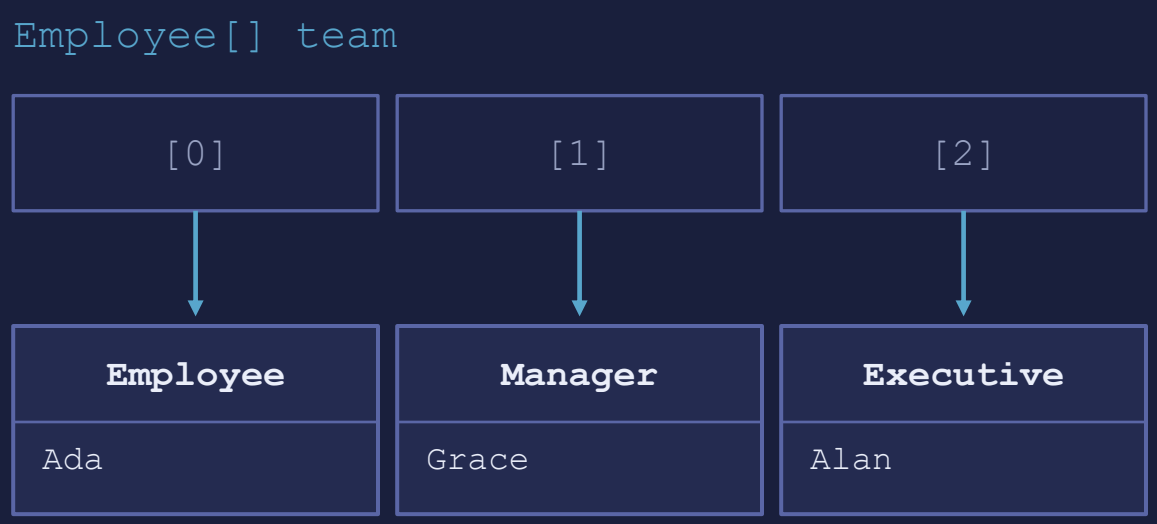
The compiler picks an overload by declared types. Also: private, static, and final methods, and constructors.

Run time (dynamic, late binding): overriding

The JVM picks an override by the object's actual class, while the program runs.

If behavior should depend on the object, put it in an overridden method, not in overloads.

One loop handles every kind of employee




```
for (Employee e : team) {
    total += e.getMonthlyPay();
}
```

Arrays and ArrayLists

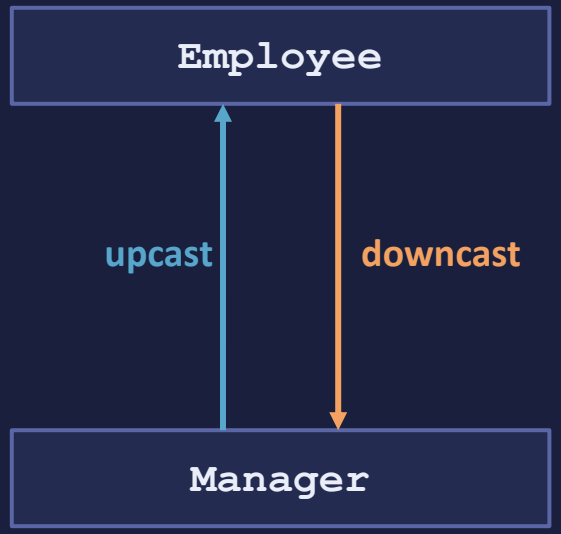
Employee[] and ArrayList<Employee> hold any mix of subclasses.

Parameters and return types

totalMonthlyPay(ArrayList<Employee> staff) works for subclasses written next year.

Note: an ArrayList<Manager> is not an ArrayList<Employee>. Generics, Week 13.

Upcasts are automatic; downcasts are checked at run time




```
Employee e = grace;                 // upcast: automatic
Manager m = (Manager) e;           // downcast: a promise
```

```
Employee ada = new Employee("Ada", 60000);
Manager oops = (Manager) ada;
```

Compiles, then throws ClassCastException

```
HourlyEmployee h = (HourlyEmployee) grace;  // sibling
```

Does not compile: incompatible types

A cast changes the declared type you use to look at an object. It never changes the object.

Test with instanceof before you cast

```
for (Employee e : staff) {
    if (e instanceof Manager m) {   // test and cast
        m.setBonus(m.getBonus() + 1000);
    }
}
```

Warning sign

A chain of instanceof tests that decides how to compute something belongs inside the classes as an overridden method. Use instanceof to reach a feature only one subclass has.

True for subclasses

An Executive passes instanceof Manager, so Alan gets the raise too.

False for null

null instanceof Manager is false, never an exception.

Pattern matching

Since Java 16, the test can declare m in one step.

instanceof checks compatibility; getClass checks the exact class

```
Employee boss = new Executive("Alan", 150000, 30000, 24000);
boss instanceof Manager                 // true
boss.getClass() == Manager.class        // false
boss.getClass().getSimpleName()         // "Executive"
```

`x instanceof T`

- Subclasses count as a match
- null gives false
- Use before a downcast, and in most type checks

`x.getClass() == T.class`

- Only the exact class matches
- null throws NullPointerException
- Use when subclasses must not match, as in some equals methods

Employee.equals uses instanceof and is final: the same ID means the same employee, for every subclass.

NEXT

Next: build it in Example 2

1. Add Executive extends Manager, and a final equals to Employee.
2. Put all three kinds of employee in an array and an ArrayList.
3. Predict both badges before you run printBadge.
4. Cast safely with instanceof; then make one cast fail on purpose.

Read the Part 2 notes and Core Java, Volume I, Sections 5.1 and 5.2.